

Conduire un audit technique efficace

La méthode, les mesures qui comptent, les pièges — et les référentiels à avoir en tête.
Tiré d'un audit réel d'application legacy, y compris de ce qui a dû y être corrigé.

1 Ce qu'un audit est, et ce qu'il n'est pas

Un audit n'est pas une revue de code. C'est un **état des lieux qui prépare une décision**, souvent un investissement. Le lecteur n'est pas un développeur : c'est celui qui signe.

Le diagnostic avant travaux. L'expert ne dit pas « abattez ce mur ». Il dit : voilà la structure, voilà les fissures, voilà ce qui arrive si vous ajoutez un étage. La décision reste au propriétaire.

L'AUDIT FAIT

- Conster, avec des preuves
- Expliquer le mécanisme d'un problème
- Chiffrer quand c'est possible
- Hiérarchiser par impact métier
- Dire ce qui est sain et à garder

L'AUDIT NE FAIT PAS

- Proposer la solution cible
- Choisir une technologie
- Chiffrer la refonte
- Lister tout ce qui déplaît
- Juger ceux qui ont écrit le code

LA RÈGLE QUI STRUCTURE TOUT

Qualifier les problèmes assez précisément pour que la solution devienne évidente — **sans jamais l'écrire**. « Le service ne peut pas être répliqué » est un constat. « Il faut passer aux jetons JWT » est une décision qui ne vous appartient pas.

2 Trier les signaux avant de commencer

Le commanditaire donne des symptômes. Ils ne sont jamais de même nature, et cette distinction détermine votre méthode de travail.

NATURE	EXEMPLE	VOTRE MÉTHODE	LE RISQUE
Constaté	« C'est lent », « c'est dur à maintenir »	Vous remontez du symptôme vers sa cause dans le code	Décrire sans expliquer pourquoi
Anticipé	« Et si on double le trafic ? »	Rien n'est observable. Vous descendez de l'architecture vers un scénario	Affirmer sans démontrer

POUR TOUT RISQUE ANTICIPÉ : ÉCRIRE UN SCÉNARIO

C'est ce qui transforme une inquiétude en risque argumenté. Nommez le moment, la charge, l'utilisateur, et déroulez ce que fait le code. « Un vendredi de pic, LiVrai ajoute deux serveurs. Une gestionnaire se connecte sur le serveur 1, sa requête suivante part sur le serveur 3, qui ne connaît pas sa session : elle est déconnectée en pleine saisie. » Personne ne débloque un budget sur une opinion.

3 Le déroulé en six temps

1 Cadrer

Reformulez le besoin avec vos mots. Écrivez noir sur blanc ce qui est **hors périmètre** et ce à quoi vous n'avez pas accès — c'est ce qui vous protège en soutenance.

2 Explorer le fonctionnel

Qui sont les utilisateurs, que peuvent-ils faire. Restez au niveau des fonctionnalités, jamais des clics.

3 Lancer l'application

Le point que tout le monde saute. Voir **SECTION 4**.

4 Analyser l'architecture

Couches, technologies, versions. Rapportez le réel au modèle attendu : ce qui manque est souvent plus parlant que ce qui est là.

5 Mesurer

Une seule mesure vaut dix paragraphes. Voir **SECTION 5**.

6 Hiérarchiser et conclure

Reprenez chaque signal initial : confirmé ou non, et par quel mécanisme. Le commanditaire doit retrouver ses mots dans votre conclusion.

4 Lancez l'application. Toujours.

Lire le code donne des hypothèses. L'exécuter donne des **preuves**. Sur l'audit dont ce document est tiré, faire tourner l'application a changé la nature de la moitié des constats.

HYPOTHÈSE ISSUE DU CODE	CE QUE L'EXÉCUTION A PRODUIT
« Le bouton Refuser pointe vers la mauvaise route »	HTTP 405 Method Not Allowed , statut inchangé en base. Le refus d'une commande était impossible en production.
« Cette page ne vérifie pas le rôle »	Un compte client a listé tous les clients avec leurs emails et créé un utilisateur . Fuite de données démontrée.
« Il n'y a pas de validation des entrées »	Volume -99 accepté et stocké. Volume abc → HTTP 500 exposé à l'utilisateur.

Une capture d'écran vaut aussi un paragraphe : sur cet audit, un seul écran montrait simultanément trois anomalies distinctes.

5 Cherchez le chiffre qui ferme le débat

Un audit contient beaucoup de qualitatif. Une seule mesure bien choisie le change de statut. Cherchez le **rapport entre deux limites de configuration** : c'est là que se cachent les seuils de rupture.

EXEMPLE RÉEL – LA MESURE QUI A PORTÉ L'AUDIT

`maxThreads = 150` côté serveur d'application, `max_connections = 151` côté base. Chaque thread retenant sa connexion sans jamais la fermer : **150 connexions possibles pour 151 autorisées, soit une marge d'une seule connexion**. La panne survient donc *avant* toute limite de processeur ou de mémoire — ce qui la rend invisible à la supervision habituelle.

CE QUI SE MESURE VITE, ET PAIE

- **Connexions ouvertes à la base** (table des processus du SGBD) → fuite de ressources et plafond réel
- **Poids d'une page à volume réaliste** (injecter N lignes, mesurer) → dégradation par la volumétrie
- **Limites de configuration** (fichiers de conf, variables du SGBD) → seuils de saturation calculables
- **Mémoire sous charge concurrente** (N requêtes en parallèle) → marge avant épuisement

COMPAREZ TOUJOURS À UN POINT DE RÉFÉRENCE

« 15,6 Mo » ne dit rien seul. « 15,6 Mo pour 20 000 livraisons, contre 3 Ko pour 4 » montre une progression linéaire sans plafond. C'est la comparaison qui fait la démonstration.

6 Les cinq pièges

PIÈGE	CE QUE ÇA DONNE	LE RÉFLEXE
Écrire la refonte	« Il faut migrer vers Spring Boot » dans un état des lieux	Décrire le mécanisme, laisser conclure
Tout mettre au même niveau	Un nommage discutable à côté de mots de passe en clair	Classer par impact métier, pas par gêne
Rester technique	« Requête non optimisée » — le commanditaire ne réagit pas	Traduire : « 3 s d'attente, 50 fois par jour »
Ne voir que le négatif	Un rapport à charge, que personne n'utilise	Nommer ce qui est sain : c'est ce qu'on garde
Affirmer sans vérifier	Un constat faux décrédibilise tout le reste	Voir ci-dessous

LE PIÈGE LE PLUS CÔUTEUX – VÉCU SUR CET AUDIT

Le schéma SQL ne déclarait aucun index : j'ai écrit « aucun index sur la table ». C'était **faux**. Le SGBD crée automatiquement un index pour toute clé étrangère — `SHOW INDEX` l'a montré. Le constat exact était plus fin : l'index existe sur la clé étrangère, mais pas sur la colonne réellement utilisée pour trier.

La leçon : ce que le code ne dit pas, l'infrastructure le fait peut-être pour lui. Vérifiez sur le système qui tourne, pas sur le fichier qui le décrit. Un seul constat faux fait douter de tous les autres.

7 Avant de rendre — sept contrôles

- Chaque signal du commanditaire est traité : **retrouve-t-il ses mots dans ma conclusion ?**
- Chaque constat est rattaché à une preuve : fichier, ligne, mesure ou capture
- Aucune solution ne s'est glissée dans le texte : **ai-je écrit « il faut » quelque part ?**
- Les points forts sont nommés : qu'est-ce qui mérite d'être conservé ?
- Les limites de l'audit sont écrites : ce que je n'ai pas pu vérifier est-il dit ?
- Les références internes se résolvent : les renvois pointent-ils vers du réel ?
- Les schémas ont une légende et se comprennent sans le texte

DISTINGUEZ TROIS NATURES DE PROBLÈMES

C'est ce qui rend un rapport actionnable. **Corrigeable tout de suite** (un bug en production, indépendant de l'architecture) · **structurel** (découle de la conception, ne se corrige pas ponctuellement) · **obsolescence** (impose une décision de trajectoire). Les premiers ne doivent pas attendre la refonte — dites-le.

8 Les référentiels : ce qu'une architecte doit en savoir

Personne ne vous demandera de réciter une norme. Ce qu'on attend de vous, c'est de **savoir laquelle emprunter, pour quoi faire, et quand elle est hors sujet**. Un référentiel cité à tort décrédibilise plus qu'il ne rassure.

LE RÉFLEXE

Un référentiel sert à **structurer et faire dialoguer**, jamais à impressionner. S'il n'apporte ni vocabulaire commun, ni ordre de travail, ni argument opposable — ne le citez pas.

ISO/IEC 25010 – LE VOCABULAIRE DE LA QUALITÉ

C'est la norme la plus directement utile en audit. Elle découpe la qualité d'un logiciel en **huit caractéristiques**, ce qui vous donne une grille pour classer vos constats — et surtout pour prouver que vous n'avez rien oublié.

CARACTÉRISTIQUE	LA QUESTION QU'ELLE POSE	SUR UN AUDIT TYPE
Performance	Temps de réponse, ressources, capacité	Poids de page, connexions, mémoire
Fiabilité	Disponibilité, tolérance aux pannes	Point de défaillance unique, réplication
Sécurité	Confidentialité, intégrité, traçabilité	Mots de passe, autorisation, secrets
Maintenabilité	Modularité, réutilisabilité, testabilité	Couche métier absente, aucun test
Compatibilité	Interopérabilité, coexistence	Socle en fin de vie, API absente
Utilisabilité	Apprentissage, protection contre l'erreur	Échecs silencieux, pas de retour visuel
Adéquation fonct.	Complétude, exactitude, pertinence	Fonction annoncée mais inopérante
Portabilité	Installation, adaptabilité, remplaçabilité	Secrets en dur, recompilation par environnement

CE QUE ÇA CHANGE CONCRÈTEMENT

« C'est mal fait » n'est pas opposable. « **Défaut de maintenabilité — sous-caractéristique testabilité** » l'est : c'est un manque nommé dans une norme internationale, que personne ne peut réduire à votre opinion. La norme ne trouve rien à votre place ; elle rend vos trouvailles discutables sur le fond plutôt que sur la forme.

TOGAF – UN CADRE D'ENTREPRISE, PAS UNE MÉTHODE D'AUDIT

Sachez ce que c'est pour savoir quand ne pas l'appliquer. TOGAF est un cadre d'architecture d'*entreprise* : il pilote la transformation d'un système d'information entier, sur des mois, avec une gouvernance. Le dérouler intégralement pour une application de quelques milliers de lignes est une erreur de proportion — et un jury le relèvera.

Son cœur est l'**ADM** (*Architecture Development Method*), un cycle en phases A à H. Deux choses à en retenir :

- **La phase B décrit l'existant avant la cible.** TOGAF nomme *Baseline Architecture* ce que vous produisez dans un audit, et *Target Architecture* ce qui vient après. Le **Gap Analysis** est l'écart entre les deux. C'est exactement la frontière que vous devez tenir : l'audit s'arrête à la baseline.
- **Les quatre domaines** — métier, données, applicatif, technique — constituent une grille de complétude utile même hors TOGAF : ai-je regardé les quatre, ou seulement le code ?

LA PHRASE JUSTE EN SOUTENANCE

« Cet audit correspond à la phase B de l'ADM : il établit la *baseline*. Le *gap analysis* et la cible relèvent de l'étape suivante.
» Vous montrez que vous situez votre travail dans un cadre reconnu **sans prétendre l'avoir appliqué**.

LES AUTRES À CONNAÎTRE DE NOM

- **ISO/IEC/IEEE 42010** — une architecture se décrit par *points de vue*, chacun répondant aux préoccupations d'une *partie prenante*. Justifie qu'un rapport comporte plusieurs vues plutôt qu'une seule.
- **Modèle 4+1** KRUCHTEN — logique, processus, développement, physique, reliés par des cas d'usage. Aide à voir quel schéma manque.
- **C4** BROWN — contexte → conteneurs → composants → code. Plus léger qu'UML quand le lecteur n'est pas technicien.
- **ISO/IEC 27001** — management de la sécurité de l'information. Dès que l'audit touche des données personnelles ou le RGPD.
- **ADR** — un fichier court par décision : contexte, choix, conséquences. **La pratique la plus rentable du lot** : tracer le *pourquoi*, pas seulement le *quoi*.

TROIS RÉFLEXES QUI VALENT PLUS QUE LES NORMES

1 Tout choix a un coût

Une architecture n'est jamais « bonne » dans l'absolu : elle arbitre entre des qualités qui s'opposent — sécurité contre simplicité, performance contre maintenabilité, découplage contre nombre de fichiers. Nommez le prix de ce que vous préconisez, sinon vous vendez du rêve.

2 Le contexte prime sur la règle

Le même pattern est excellent à 100 utilisateurs et absurde à 3. Un référentiel appliqué sans être confronté à la taille du système, au budget et à la maturité de l'équipe produit de la sur-ingénierie — et TOGAF sur un CRM de treize classes en est l'exemple type.

3 Écrivez pour celui qui décide

Votre rapport est lu par quelqu'un qui engage de l'argent, pas par un pair. Traduisez systématiquement le technique en conséquence métier. C'est là que se joue votre crédibilité, bien plus que dans les normes citées.